Milestone 2. Core functionalities Development

Activity 2.1 Develop the core functionalities:

Generate Structured Comic Panel Outline

Function: generate_outline()
File: gemini_flash.py

- This function uses the **Gemini Flash** AI model to generate a structured 5-panel comic outline based on the user's story prompt.
- Each panel contains a panel number, title, scene description, and an image generation prompt.
- The output is a list of dictionaries, one for each comic panel.

```
def generate_outline(user_prompt: str) -> list:
    """
    Generates a 5-panel comic layout based on the user's story idea using Gemini.

    Args:
        user_prompt (str): The user's comic idea prompt.

    Returns:
        list: A list of dictionaries, one for each panel.
    """
    prompt = f"""
You are a professional AI comic planner.

Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions for a comic based on the story idea below:

STORY: "{user_prompt}"

Each JSON object must include:
- "panel" (integer)
- "title" (string)
- "scene_description" (string)
- "image_prompt" (string)

Respond ONLY in this valid JSON format, without any explanations or markdown:
[
  {{
    "panel": 1,
    "title": "Title here",
    "scene_description": "Scene description here",
    "image_prompt": "Image prompt for Stable Diffusion"
  }},
  ...
]
"""
```

```python
try:
    response = model.generate_content(prompt)
    output_text = response.text.strip()

    print("\n🔥 RAW GEMINI RESPONSE 🔥\n", output_text)

    # Remove any markdown formatting if present
    if output_text.startswith("```json"):
        output_text = output_text.replace("```json", "").replace("```", "").strip()

    panel_data = json.loads(output_text)

    # Additional structure validation
    if not isinstance(panel_data, list):
        raise ValueError("Gemini response is not a list.")

    for panel in panel_data:
        if not isinstance(panel, dict) or not all(key in panel for key in ("panel", "title", "scene_description", "image_prompt")):
            raise ValueError(f"Invalid panel format or missing keys: {panel}")

    return panel_data

except json.JSONDecodeError as e:
    print("❌ JSON Decode Error:", e)
    print("❌ Full Text Received:\n", output_text)
    return [{"error": f"JSON parsing failed: {str(e)}"}]

except Exception as e:
    print("❌ Unexpected Error:", e)
    return [{"error": f"Generation failed: {str(e)}"}]
```

Generate Comic Story Narration and Dialogue

Function: generate_story()
File: gemini_pro.py

- This function uses the **Gemini Pro** AI model to expand the panel outlines into a full comic-style story.
- It creates engaging narration and character dialogues for each panel.
- The output is a single formatted text containing all the panels' stories.

```python
def generate_story(outline: list) -> str:
    """
    Generates a detailed comic story with narration and character dialogue
    from a list of comic panel outlines using Gemini 1.5 Pro.

    Args:
        outline (list): A list of strings representing each comic panel's idea.

    Returns:
        str: The generated comic story text or an error message.
    """

    # Format the panel outline as a numbered list for clarity
    formatted_outline = "\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])

    # Construct the prompt
    prompt = f"""
You're a comic book writer.

Given the following panel breakdown, write a comic-style story with engaging narration and character dialogues for each panel.

Panel Outline:
{formatted_outline}

Guidelines:
- Use a fun and engaging tone, like an actual comic book.
- Include narration and clearly marked character lines.
- Keep each panel self-contained but part of a cohesive story.
"""

    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"
```

Generate Comic Illustrations from Prompts

Function: generate_image()
File: image_generator.py

- This function uses the **Stable Diffusion** model to create a comic-style image based on the provided image prompt.
- It sanitizes the prompt for safe filenames and saves the generated image to the static/panels directory.
- Returns the file path to the saved image.

```python
def generate_image(prompt, filename=None):
    if not filename:
        filename = sanitize_filename(prompt)

    image = pipe(prompt).images[0]
    path = f"static/panels/{filename}"
    os.makedirs(os.path.dirname(path), exist_ok=True)
    image.save(path)
    return path
```

Organize Comic Panels into Layout Structure

Functions: build_comic_layout()

File: layout_builder.py

- This function organizes the generated images and full comic story into a structured layout.
- It matches each image with its corresponding panel story.
- The output is a list of dictionaries containing the panel number, image path, and text.

```python
def build_comic_layout(image_paths, full_story, outline):
    # Split the full story into individual panel segments
    story_panels = full_story.split("**Panel")
    story_panels = [f"**Panel{panel}" for panel in story_panels if panel.strip()]

    # Build layout panel by panel
    layout = []
    for idx, (image, text, panel_info) in enumerate(zip(image_paths, story_panels, outline), start=1):
        layout.append({
            "panel": idx,
            "title": panel_info.get("title", f"Panel {idx}"),
            "image_path": image,
            "text": "\n".join(text.strip().splitlines()[1:]).strip(),  # remove panel title
            "scene_description": panel_info.get("scene_description", "")
        })

    return layout
```

Export Comic Panel and Story into PDF

Function: save_pdf()
File: exporters.py

- This function compiles the full comic into a multi-page **PDF file** using the FPDF library.
- Each panel's image and narration are placed neatly on separate pages.

- The final PDF file is saved into the static/exports folder, and the path is returned.

```python
def save_pdf(layout):
    pdf = PDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    # Add Unicode font
    pdf.add_font('DejaVu', '', FONT_PATH, uni=True)
    pdf.set_font('DejaVu', '', 12)

    for panel in layout:
        image_path = panel['image_path']
        story_text = panel['text']

        pdf.add_page()

        # Panel title (no bold)
        pdf.set_font('DejaVu', '', 14)
        pdf.cell(0, 10, f"Panel {panel['panel']}", ln=True, align="C")
        pdf.set_font('DejaVu', '', 12)

        # Image placement
        y_image = 30
        image_height = 100
        spacing_after_image = 15

        if os.path.exists(image_path):
            pdf.image(image_path, x=10, y=y_image, w=pdf.w - 20, h=image_height)
        else:
            pdf.set_y(y_image)
            pdf.multi_cell(0, 10, f"Image missing: {image_path}")

        # Text placement below image
        pdf.set_y(y_image + image_height + spacing_after_image)
        story_lines = story_text.strip().splitlines()
        # Remove title line like "**Panel 1: Title**" if present
        if story_lines and story_lines[0].strip().lower().startswith("**panel"):
            story_lines = story_lines[1:]
        cleaned_text = "\n".join(story_lines).strip()
        pdf.multi_cell(0, 10, cleaned_text)

    # Save with timestamp
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    pdf_path = os.path.join(EXPORT_FOLDER, filename)
    pdf.output(pdf_path)

    return pdf_path
```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in **routes.py**.
Each route is linked to the corresponding backend function that processes the comic generation workflow developed in Activity 2.1.

Process User Input

- **HTML Form**: Created in index.html
- **Users enter**: story prompt, main character name, setting, tone, and art style
- **FastAPI captures inputs** using Form(...) parameters from the request
- **Pydantic schema** (PromptRequest) is used to validate JSON input for API-based comic generation

Integrate Gemini API Calls

Each route handler connects directly to the respective AI-powered function modules:

- **/generate** →
  - Calls generate_outline() to create panel outlines from user prompt
  - Calls generate_story() to expand into narration and dialogue
  - Calls generate_image() for each panel to create comic-style images
  - Calls build_comic_layout() to organize the final layout
  - Calls save_pdf() to compile and export the complete comic into a PDF
- **/generate-comic/json** →
  - Similar flow as /generate, but triggered via a raw JSON API request
  - Returns JSON response with layout data and PDF path for API clients
- **/test-image** →
  - Calls generate_image() directly with a custom prompt to test image generation functionality separately

All AI-generated responses (text and images) are processed, cleaned, and injected into the Jinja2 frontend templates (comic_preview.html, export_success.html) for a seamless user experience.